

CRUD em Docker em Ambiente Linux

Felipe Batista dos Santos
Sistemas Operacionais II - 3ºSemestre
Prof. Nelson

1. Introdução

O seguinte projeto tem por objetivo a aprendizagem, desenvolvimento, teste e por fim a aplicação do Docker em ambiente Linux, para isso será necessário a utilização de um programa com front-end, back-end e banco de dados, Docker, e alguma forma de virtualização.

Será utilizado Java para Back-end, React para Front-end, postgresql para banco de dados, docker e WSL para a virtualização.

2. Criação do Ambiente Linux

Prompt: “Me guie para instalação de Ubuntu para virtualização pelo Windows”

Como o trabalho deve ser feito em um ambiente Linux, é preciso utilizar virtualização em computadores com ambiente Windows ou IOS. Para isso será utilizado o VirtualBox, que pode ser baixado gratuitamente pelo site oficial: [Baixar VirtualBox](#), a virtualização possui requisitos mínimos:

Windows 10 ou 11 (64-bit)

Processador com suporte a virtualização (Intel VT-x ou AMD-V) — verifique na BIOS

Pelo menos **8 GB de RAM** no sistema (será alocado no mínimo 2 GB para a VM)

Pelo menos **25 GB livres** em disco

Virtualização habilitada na BIOS/UEFI

Após a instalação, é preciso baixar também a BIOS da ambiente Linux que desejado, nesse projeto foi escolhido o Ubuntu, por ser um dos Sistemas Operacionais Linux mais conhecidos, com mais tutoriais, alta performance e leveza. [Baixar Ubuntu](#)

Depois no VirtualBox deve criar uma nova snapshot e colocar o ambiente Ubuntu, após a instalação fazer a seguinte configurações :(atualizações, Guest Additions para melhor desempenho.

Outra solução é utilizar o WSL2 - Windows Subsystem for Linux 2, que roda um **kernel Linux real dentro do Windows**, usando uma virtualização leve. O projeto será feito com WSL2 por ser mais fácil, leve e rápido do que a virtualização comum.

Requisitos Mínimos Identificados

Componente de Hardware	Requisito Mínimo Configurado
Processador (CPU)	Suporte à virtualização de hardware (Intel VT-x ou AMD-V) habilitada na BIOS/UEFI.
Memória RAM	Mínimo de 8 GB no host (com alocação dinâmica e leve para a instância do WSL2).
Armazenamento	Mínimo de 25 GB de espaço livre em disco sólido (SSD/HD).
Sistema Operacional	Windows 10 ou 11 (64-bit), build compatível com WSL2.

3. Exploração do Sistema de Arquivos Linux

Antes do provisionamento dos serviços do projeto, realizou-se uma varredura exploratória da árvore estrutural do Linux para compreender a organização de diretórios padrão (FHS - Filesystem Hierarchy Standard):

- **`/` (Raiz):** Diretório soberano do sistema, onde todas as outras ramificações se originam.
- **`/home`:** Espaço reservado para o armazenamento dos dados e configurações pessoais dos usuários comuns.
- **`/etc`:** Centralização de arquivos de configuração estáticos do sistema operacional e de serviços instalados.
- **`/var`:** Armazena dados variáveis que tendem a crescer durante o tempo de execução, como logs do sistema (`/var/log`) e arquivos temporários de banco de dados.
- **`/usr`:** Diretório que contém utilitários, bibliotecas e binários de programas voltados aos usuários.
- **`/tmp`:** Espaço destinado à gravação de arquivos temporários criados por processos em execução, limpos rotineiramente na reinicialização.

Comandos Essenciais Praticados via Terminal

Os seguintes comandos fundamentais de manipulação e diagnóstico foram executados na preparação do ambiente de desenvolvimento do projeto:

- **`cd` (Change Directory):** Utilizado para navegar de forma ágil entre as estruturas de pastas do terminal Linux.
- **`mkdir -p`:** Criação de novos diretórios. O uso da flag `-p` garante a criação de toda a árvore hierárquica informada sem disparar erros caso as pastas já existam.
- **`ls -la`:** Listagem minuciosa de conteúdo de pastas, exibindo formatos detalhados (permissões, donos, tamanhos) e incluindo arquivos ocultos.
- **`chmod`:** Altera as permissões de acesso a arquivos e pastas utilizando notação simbólica ou numérica (ex: leitura, escrita, execução).
- **`sudo`:** Elevação temporária de privilégios para execução de comandos com poder de superusuário (root).
- **`kill`:** Comando de gerenciamento de processos, responsável pelo encerramento forçado de tarefas através do seu identificador numérico (PID).

4. Instalação e Configuração do Docker

A instalação do ambiente Docker no ecossistema WSL2 Ubuntu foi conduzida via linha de comando seguindo o padrão oficial da Docker Engine, adicionando os repositórios seguros e as chaves GPG necessárias para atualizações contínuas:

```
# Atualização de índices e instalação de pré-requisitos
sudo apt-get update
sudo apt-get install -y ca-certificates curl

# Configuração e inserção da chave oficial GPG do Docker
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Adição do repositório estável às fontes do APT
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Instalação completa da Docker Engine e utilitários de build
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

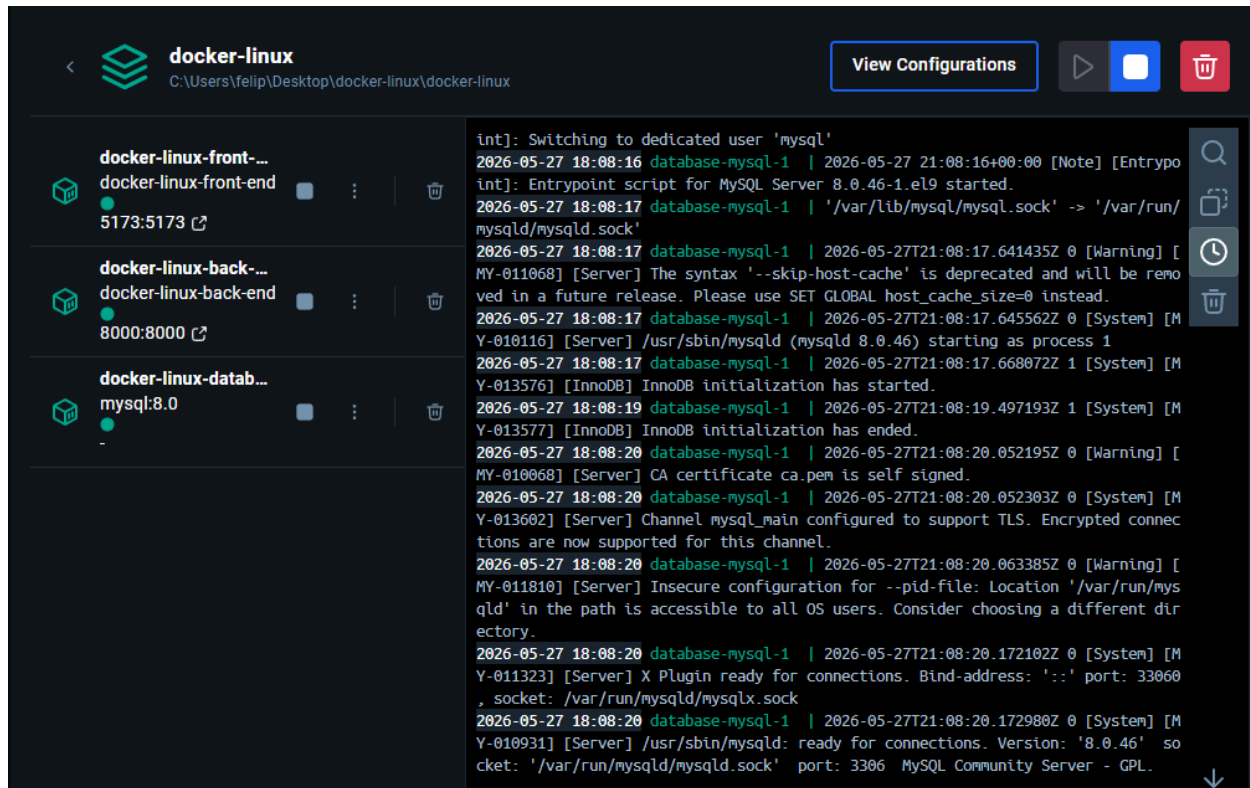
Configuração Pós-Instalação: Execução sem `sudo`

Por padrão, o daemon do Docker interage exclusivamente através de sockets Unix pertencentes ao usuário root, exigindo a inserção do comando `sudo` para cada requisição. Para sanar essa barreira e otimizar o fluxo de trabalho, o usuário ativo foi vinculado ao grupo de segurança do Docker:

```
sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
```

Dessa forma, o usuário ganha privilégios automáticos para gerenciar containers de forma direta. O status do serviço foi verificado e validado com o utilitário de testes padrão:

```
sudo service docker start
sudo service docker status
docker run hello-world
```



5. Conceitos Docker na Prática

Docker é uma plataforma open-source de containerização, desde do empacotamento, distribuição e execução deles.

Os containers são uma forma de virtualização, seu propósito veio do problema das máquinas virtuais, nas quais simulavam toda uma máquina, sendo necessário emular hardware e sistemas operacional inteiros, o que acabava consumindo muito poder de processamento da CPU.

Com isso os contêineres vieram como uma forma de substituí-los, já que isolam sistemas/programas sem ter que recriar toda a máquina, pois eles compartilham o kernel do Sistema operacional host, e isolam o processo por Namespaces: Isolar o que o container pode ver e cgroups: limita o que o container pode ver

Imagens são como se fossem um manual de instruções e especificações dos containers, a partir deles se instancia os containers.

Volume: Para a utilização de dados em containers e não perdê-los caso sejam removidos

6. Dockerfiles - Análise e Explicação

Definindo o tema do projeto

Tema e Funcionalidades O projeto consiste em um **Sistema de Gerenciamento de Tarefas (To-Do List)**. O sistema aplica rigorosamente o padrão CRUD, permitindo ao usuário: cadastrar novas tarefas (Create), listar e buscar atividades (Read), editar informações e atualizar o status (Update), e deletar registros indesejados (Delete).

Stack Tecnológica e Arquitetura

A infraestrutura foi dividida em três camadas containerizadas e independentes:

- **Banco de Dados (MySQL):** Armazenamento relacional estruturado em duas tabelas principais: Usuarios (credenciais de acesso) e Tarefas (dados da atividade com chave estrangeira para o usuário).
- **Back-end (Python com FastAPI):** Responsável por fornecer uma API REST de alta performance que gerencia a lógica de negócio e processa as requisições HTTP do sistema.
- **Front-end (React):** Interface de usuário (UI) dinâmica e reativa, responsável por consumir a API e exibir os formulários e listas de tarefas no navegador.



Dockerfile do Back-end (Python / FastAPI)

```
# Usa uma imagem oficial e leve do Python baseada em Debian Slim
FROM python:3.10-slim

# Define o diretório interno de trabalho para os comandos
subsequentes
WORKDIR /app

# Copia a listagem de dependências lógicas para reaproveitamento de
cache
COPY requirements.txt .

# Instala as bibliotecas de desenvolvimento sem salvar pacotes
temporários
RUN pip install --no-cache-dir -r requirements.txt

# Transfere a totalidade do código-fonte local para o container
COPY . .

# Expõe e documenta logicamente a porta padrão de rede
EXPOSE 8000
```

```
# Executa o servidor ASGI Uvicorn escutando todas as interfaces de rede
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile do Front-end (React / Vite)

```
# Baseia-se no Node.js sob a distribuição Alpine Linux (foco em leveza)
FROM node:18-alpine

# Estabelece a pasta interna de execução
WORKDIR /app

# Copia os manifestos de pacotes Javascript
COPY package.json package-lock.json* ./

# Processa a instalação limpa de dependências do ecossistema React
RUN npm install

# Transfere os arquivos de design e componentes para o ambiente isolado
COPY . .

# Expõe logicamente a porta de desenvolvimento padrão do Vite
EXPOSE 5173

# Comanda a execução do servidor Vite aberto para tráfego externo
CMD ["npm", "run", "dev", "--", "--host"]
```

7. Docker Compose - Orquestração

Para gerenciar de forma unificada as múltiplas camadas da aplicação sem a necessidade de comandos manuais massivos para cada componente, utilizou-se o **Docker Compose**. O manifesto `docker-compose.yml` centraliza a declaração de portas, variáveis de ambiente de conexão, políticas de reinicialização automática, volumes nomeados e isolamento de redes customizadas.

```
version: '3.8'

services:
  database-mysql:
    image: mysql:8.0
    container_name: docker-linux-database-mysql-1
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root_password
```



```
    MYSQL_DATABASE: DockerLinux
    MYSQL_USER: my_user
    MYSQL_PASSWORD: my_password
ports:
  - "3306:3306"
volumes:
  - mysql-data:/var/lib/mysql
  - ./init.sql:/docker-entrypoint-initdb.d/init.sql
networks:
  - rede-app
```

```
back-end:
  build: ./backend
  container_name: docker-linux-back-end-1
  ports:
    - "8000:8000"
  environment:
    - DB_HOST=database-mysql
    - DB_USER=my_user
    - DB_PASSWORD=my_password
    - DB_NAME=DockerLinux
  depends_on:
    - database-mysql
  networks:
    - rede-app
```

```
front-end:
  build: ./frontend
  container_name: docker-linux-front-end-1
  ports:
    - "5173:5173"
  depends_on:
    - back-end
  networks:
    - rede-app
```

```
volumes:
  mysql-data:
```

```
networks:
  rede-app:
    driver: bridge
```

Decisões Críticas de Arquitetura: A diretiva `depends\_on` garante a ordem correta de inicialização do ecossistema. A comunicação interna ocorre de forma isolada pela rede em driver `bridge` chamada `rede-app`. Notavelmente, o container de back-end resolve o endereço de acesso ao banco chamando a string textual `database-mysql` (nome do serviço), delegando ao DNS embutido do Docker a tradução para o IP mutável interno.

8. Banco de Dados em Container

O container de armazenamento foi provisionado a partir da imagem corporativa oficial do `mysql:8.0`. Para mitigar o comportamento efêmero natural dos containers e assegurar que os registros persistam, mapeou-se o volume nomeado `mysql-data` apontando diretamente para o diretório interno `/var/lib/mysql`.

Adicionalmente, o script de estruturação SQL inicial (`init.sql`) foi injetado na pasta de inicialização automática do container (`/docker-entrypoint-initdb.d/`). Esse procedimento garante o provisionamento automático do banco de dados relacional e a criação das tabelas na primeira subida da stack infraestrutural:

```
DROP DATABASE IF EXISTS DockerLinux;
CREATE DATABASE DockerLinux;
USE DockerLinux;

CREATE TABLE Tarefas (
    id INT PRIMARY KEY AUTO_INCREMENT,
    titulo VARCHAR(100) NOT NULL,
    descricao VARCHAR(255) NOT NULL,
    status VARCHAR(50) NOT NULL,
    data_criacao DATETIME DEFAULT CURRENT_TIMESTAMP,
    data_atualizado DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

CREATE TABLE Usuarios (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL
);

CREATE TABLE Tarefas_Usuarios (
    idTarefa INT,
    idUsuario INT,
    PRIMARY KEY (idTarefa, idUsuario),
    FOREIGN KEY (idTarefa) REFERENCES Tarefas(id) ON DELETE CASCADE,
    FOREIGN KEY (idUsuario) REFERENCES Usuarios(id) ON DELETE
CASCADE
);
```

9. Evidências do CRUD Funcionando

Com a infraestrutura ativa e operacional, realizou-se a validação ponta a ponta do fluxo de dados. O front-end em React consome as rotas expostas pela API REST do FastAPI através de requisições de rede assíncronas utilizando o método global `fetch()`. O back-end recebe as chamadas, valida os esquemas de dados e interage com o banco MySQL via conexões SQL

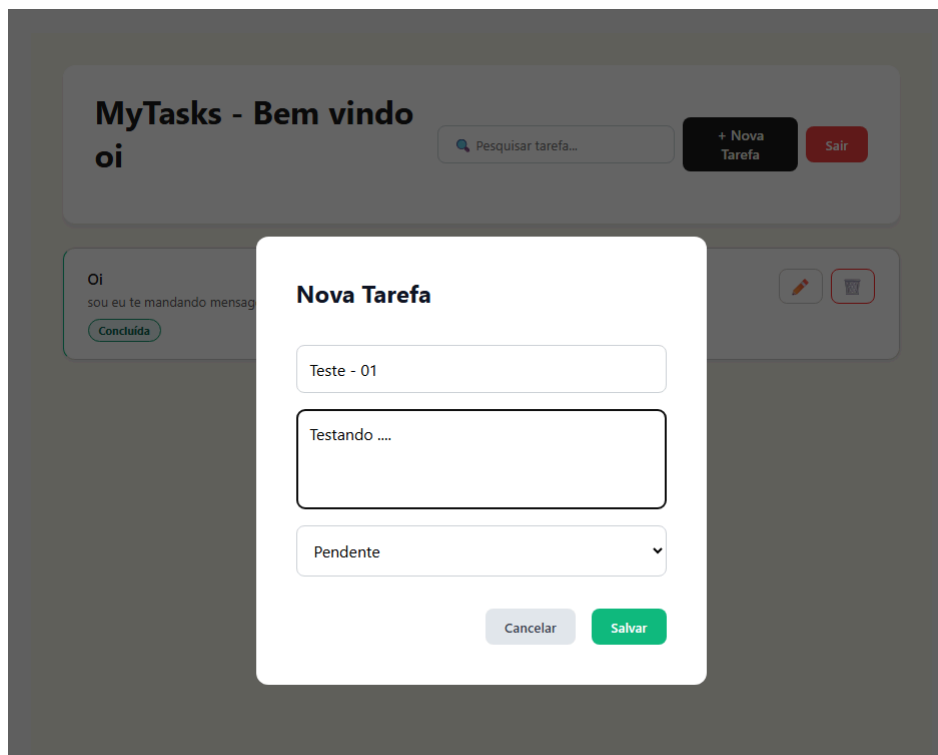
estruturadas.

O sistema atende perfeitamente aos requisitos práticos exigidos de manipulação de dados em uma entidade, operando com respostas imediatas sem travamentos na interface.

INTERFACE WEB DO SISTEMA

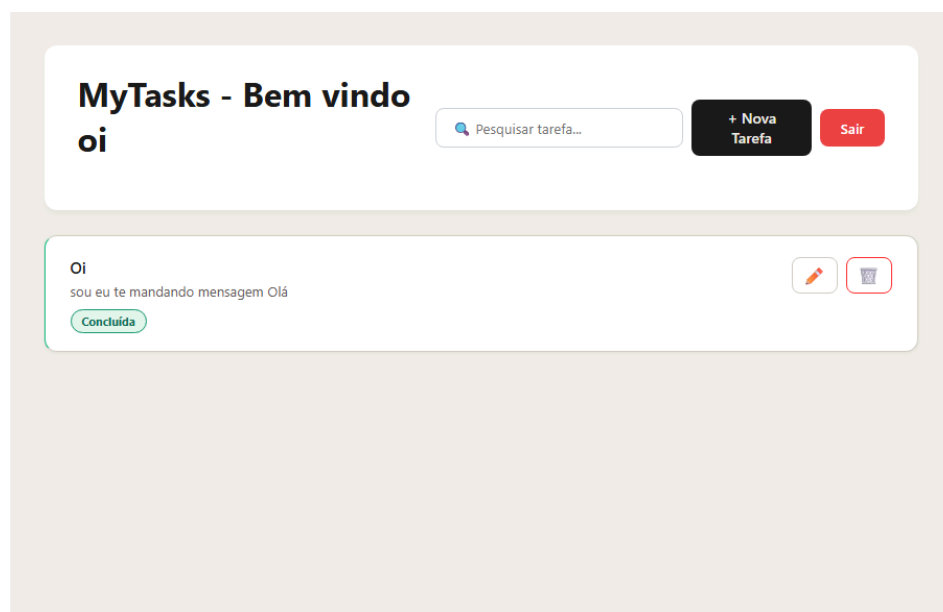


Update





DELETE



DOCUMENTAÇÃO FASTAPI (SWAGGER)]

default		^
GET	/login Login	▼
POST	/cadastro Cadastrar Usuarios	▼
GET	/tarefas Get All Tarefas	▼
POST	/tarefas Cadastrar Tarefa	▼
GET	/tarefas/{id} Get Tarefas	▼
PUT	/tarefas/{id} Atualizar Tarefa	▼
DELETE	/tarefas/{id} Deletar Tarefa	▼
Schemas		^
HTTPValidationError > Expand all object		
ValidationError > Expand all object		

10. Persistência de Dados

Para comprovar a eficácia do mapeamento do volume relacional `mysql-data` declarado na arquitetura do Docker Compose, executou-se um teste de estresse destrutivo controlado no ambiente Linux. Primeiro, inseriram-se registros reais através dos formulários do navegador. Posteriormente, o terminal foi acionado para encerrar e remover todos os componentes ativos da aplicação:

```
# Desativa, remove os containers ativos e extingue as redes de
comunicação associadas
docker compose down
```

Em seguida, a stack foi restabelecida com o comando `docker compose up -d`. Ao recarregar a interface gráfica web, constatou-se que todas as tarefas criadas previamente mantiveram-se intactas. Isso comprova tecnicamente que os arquivos de dados brutos foram salvos externamente na pasta do host Linux, sobrevivendo à destruição física completa do container do banco.

11. Monitoramento via Terminal Linux

O diagnóstico em tempo real do ecossistema Linux containerizado foi acompanhado ativamente por comandos de telemetria integrados ao terminal, assegurando que o consumo de memória interna e processamento estivessem operando em conformidade técnica ideal.

COMANDO DOCKER PS

```
PS C:\Users\felip> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
19f57a39139f   docker-linux-front-end              "docker-entrypoint.s..." 13 minutes ago Up 13 minutes 0.0.0.0:5173->5173/tcp              docker-linux-front-end-1
0d22d621ac13   docker-linux-back-end              "uvicorn main:app --..." 14 minutes ago Up 13 minutes 0.0.0.0:8000->8000/tcp              docker-linux-back-end-1
d941f3795c47   mysql:8.0                          "docker-entrypoint.s..." About an hour ago Up 13 minutes 3306/tcp, 33060/tcp                docker-linux-database-mysql-1
```

COMANDO DOCKER STATS

```
CONTAINER ID   NAME                                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
19f57a39139f   docker-linux-front-end-1           0.00%     325.9MiB / 6.73GiB   4.73%     38.4kB / 1.68MB   0B / 0B     37
0d22d621ac13   docker-linux-back-end-1           0.17%     42.7MiB / 6.73GiB   0.62%     26.7kB / 17.2kB   0B / 0B     2
d941f3795c47   docker-linux-database-mysql-1     0.64%     383.4MiB / 6.73GiB   5.56%     8.69kB / 11.3kB   0B / 0B     39
```

12. Uso de IA no Projeto

Em boa parte do relatório foi possível ver que em todas as etapas tiveram algum uso de IA, mostrando qual foi o prompt, na maioria das vezes o seu uso é pra entender a matéria/seção e a partir disso escrevi com as minhas próprias palavras, em outros casos como imagens, diagramas foi usado o que a IA fez sem alterações, e também para reescrever o texto pra ter uma escrita melhor.

A IA foi utilizada como uma forma de suporte, para aprendizado, testes, tirar dúvidas e agilização de processos

13. Dificuldades e Soluções

Durante o processo tive algumas dificuldades com tecnologias na quais não possuía conhecimento como docker, FastAPI e React. Por exemplo ao utilizar o FastAPI não sabia como fazer para mandar os comandos SQL para o Banco de dados, então para isso foram usados documentos e vídeos, e a IA para confirmar se não tinha erros.

Outro ponto na qual tive dificuldade foi com o Docker, na qual eu não sabia de suas definições e como utilizá-lo, da mesma forma pedi para IA me ensinar o que é e fazer exercícios para confirmar que entendi o conteúdo.

14. Conclusão

Container é uma ferramenta extremamente poderosa, que veio como uma forma mais fácil, otimizada e rápida do que a máquina Virtual, acabando com o problema de “Na minha máquina roda”.

O Linux é o pai dos sistemas operacionais, toda a base de Windows e outros sistemas se baseiam nele, sendo muito importante nos tempos atuais, para a configurações de rede, cibersegurança, otimização de hardware e outros

15. Referências de Consulta

Uso das IAs:

<https://claude.ai>

gemini.google.com

Vídeos:

FastAPI

<https://youtu.be/gbk6SsxZVG8?si=VgcFGm7WrFZOi6oh>

Docker

<https://youtu.be/ntbplfS44Gw?si=rP0dGnhTAzi3DveJ>

<https://www.youtube.com/watch?v=DdoncfOdrU8&t=1084s>

Documentos

<https://docs.docker.com/>

<https://www.redhat.com/pt-br/topics/containers/what-is-docker>

<https://medium.com/@parth.khajgiwale/building-a-crud-api-with-fastapi-my-sql-and-postman-8e849e62d9fe>